Introduction

This brief document explain the AI-Assisted development phase of the legacy system. It focuses on relevant prompts, outputs and decisions.

We instructed the AI tool to keep track of conversation, a detailed and complete version of all the prompts used to develop the legacy system can be found in */SDMO/AI Conversation/Phase*.

A Step-by-Step approach

Instead of generating the entire system with a single prompt, we decided to decompose the system into its three main components:

- The Legacy Device
- The Edge Gateway
- The Cloud Service

And tackle the development of each component with a different prompt, the process has been incremental in order to allow an easy check of the code for each step, the relevant prompts are:

User Prompt:

I was thinking about proceeding by following the UML Class diagram and the Sequence Diagram.

** generateData() can simply be a function that generate a short random sequence of data like this: <String><TimeStamp>.*

** encryptData(string) rely on tru AES*

** sendEncryptedData() → I want to address this function in a simple way, for now, let us make this function put the generated data encrypted bu the previous fanction into a buffer: a list or dynamic array, pick the simplest.*

For now generate this component I will inspect it and then decide how to proceed with the rest

AI Response:

I have generated the `Legacy Device` component exactly as you requested based on your UML and sequence diagrams.

You can review the code here: [`src/legacy_device.py`]

I successfully tested the script, and the encryption works as expected.

Please inspect the component, and let me know how you'd like to proceed with the Edge Gateway!

User Prompt:

Ok now go on with the implementation of the Edge device as described in the Class_Diagram:Phase0.puml

** keep in mind to provide to it the same pre-shared-key of the legacy_device.*

** negotiateSession(): void --> Leave this empty for now.*

** receiveData(encyptedData) : --> must pop encryptedData from the legacy_device buffer*

** decryptData(encryptData) : --> decrypt the received data from the buffer*

** reEncryptData() --> reEncrypt with an Hard-Coded RSA key since for now negotiateSession() is empty*

** forvardData() --> Just print out the re-encrypted data for now.*

For every functionality that is left Empty put a //TODO in the code, when we will create the CloudService then we will figure it out.

AI Response:

I have successfully implemented the Edge Gateway according to your instructions and the sequence from the class diagram. I've also recorded this latest interaction into the `conversation_log.md` inside the `IA Conversation/Phase1/2026-09-11` folder.

You can inspect the code here: [`src/edge_gateway.py`]

User Prompt:

Ok and let us start now with the Cloud Service.

In order to keep it simple, I was thinking of implementing the connection in a way similar with the connection between the gateway and the legacy device. But I have concern about the key_negotiation part.

Moreover since an entire TLS handshake is quite complex since it includes things as CIPHERSUITE and moreover this entire thing will be repleced by the post quantum algorithm, I was thinking about implementing this key agreement as a Diffie-Hellman key exchange (the schoolbook version, that is even simpler than the one really used). Before proceeding with writing the code, tell me what you think about all of this, how can we implement a "fake TCP connection" ?

AI Response:

To simulate a stateful connection (like a handshake) without true network sockets, we can use direct method calls between the objects to act as the "network round-trips". Here is a simple way to do it:

1. The Handshake (Diffie-Hellman Negotiation):

Instead of sending packets, the `EdgeGateway` calls a method on the `CloudService` and passes its DH public key. The `CloudService` returns its own DH public key in response.

** `EdgeGateway` generates its DH private/public keys.*

** `EdgeGateway` calls: `cloud_public_key = cloud_service.negotiate_dh(gateway_public_key)`*

** Inside that method, the `CloudService` generates its own keys, computes the shared secret, and returns its public key.*

** The `EdgeGateway` receives the response and computes the identical shared secret.*

(Boom! Handshake complete, shared secret established.)

2. The Data Transfer:

Once the session is negotiated, we can simulate sending data exactly like we did with the legacy device.

** We can give the `CloudService` an `incoming_buffer` (a list).*

** The `EdgeGateway`'s `forwardData()` method will take the RSA-encrypted (or AES-encrypted using the new DH shared secret) data and append it to `cloud_service.incoming_buffer`.*

** The `CloudService` then pops the data from the buffer and decrypts it.*

By passing the object references to each other (e.g., `gateway.connect(cloud_service)`), they can communicate seamlessly.

Does this architecture sound good to you? If so, I can go ahead and write the implementation plan and the code for the `CloudService` and update the `EdgeGateway` to complete the chain!

As can be seen we stuck with the hint given by the track to keep the all system simple. Then no true TCP connection has been implemented in order to reduce the layers of complexity and the DH key exchange has been implemented as a simple version: no elliptic curves, shared secret computed in a trivial way. The system has been then designed to be functional and easy understandable.